École nationale
de la statistique
et de l'analyse
de l'information

Machine learning 2A — Programmation algorithmique
Travaux pratique 1 : Régression de Ridge, données synthétiques et optimisation

Année universitaire 2025 - 2026

Introduction et motivation

La régression linéaire classique (moindres carrés ordinaires, OLS) fournit des estimateurs sans biais pour les coefficients d'un modèle. Toutefois, trois phénomènes nuisent à sa performance en pratique :

Surapprentissage (overfitting). Lorsqu'un modèle est trop complexe par rapport à la quantité d'information disponible, il s'ajuste aux fluctuations aléatoires des données et généralise mal. La régularisation, notamment la régression de Ridge, combat ce problème en ajoutant un terme qui limite la taille des coefficients et stabilise les prédictions.

Multicolinéarité. Si plusieurs variables explicatives sont très corrélées, la matrice $\mathbf{X}^T\mathbf{X}$ est proche de la singularité. L'estimateur OLS devient alors instable : une petite perturbation des données peut entraîner de grandes variations des coefficients. La régularisation de Ridge ajoute un terme $\lambda\mathbf{I}$ pour rendre l'inversion bien conditionnée et réduire la variance des paramètres.

Biais–variance. La régularisation introduit du biais mais réduit fortement la variance des estimateurs. L'objectif est de trouver un compromis qui minimise l'erreur quadratique totale.

Ce travail pratique vise à vous familiariser avec l'implémentation de la régularisation de Ridge et l'utilisation de différents schémas d'optimisation (descente de gradient, moment, accélération de Nesterov/FISTA et descente stochastique). Afin d'illustrer les phénomènes évoqués ci-dessus, plusieurs jeux de données synthétiques vous sont proposés. Vous devrez ensuite analyser les performances des différents algorithmes sur ces jeux et discuter des compromis obtenus.

Régression de Ridge

Le but de la régression de Ridge est d'estimer un vecteur de coefficients $\mathbf{w} \in \mathbb{R}^d$ minimisant la fonction de perte

$$L(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \mathbf{w}^T\mathbf{x}_i\right)^2 + \lambda \left\|\mathbf{w}\right\|_2^2,$$

où $\lambda \geq 0$ est le paramètre de régularisation. Le cas $\lambda = 0$ correspond au modèle OLS. Lorsque les variables sont corrélées ou que le rapport n/d est faible, cet estimateur peut être instable et présenter une grande variance. La pénalisation L^2 stabilise l'inversion et contrôle la taille des coefficients.

Jeux de données synthétiques

Cette section présente quatre jeux de données distincts conçus pour mettre en évidence des comportements particuliers de la régression. Dans tous les cas, les observations sont notées $(\mathbf{x}_i, y_i)$ avec $\mathbf{x}_i \in \mathbb{R}^d$ et $y_i \in \mathbb{R}$. Le terme de bruit ϵ_i est supposé suivre $\mathcal{N}(0, \sigma^2)$ avec σ précisé ci-dessous.

1. **Jeu A : modèle bien conditionné.**

   Construction. On fixe $n = 1000$ observations et $d = 20$ variables. Les vecteurs $\boldsymbol{x}_i$ sont tirés indépendamment selon $\mathcal{N}(\mathbf{0}, \mathbf{I}_d)$, le vecteur de coefficients de référence $\mathbf{w}$ est tiré dans $\mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ et les bruits ϵ_i sont indépendants, centrés et de variance 5^2. Le ratio n/d élevé garantit une matrice $\mathbf{X}^T\mathbf{X}$ bien conditionnée.

   Phénomène visé. Ce jeu sert de référence : ni surapprentissage ni multicolinéarité ne doivent apparaître de manière marquée. La régularisation devrait avoir un effet marginal.

2. **Jeu B : surparamétrisation et surapprentissage.**

   Construction. On choisit un nombre de variables d nettement supérieur au nombre d'observations, par exemple $n = 100$ et $d = 200$. Les vecteurs $\boldsymbol{x}_i$ et le vrai $\mathbf{w}$ sont générés comme pour le jeu A mais la dimension élevée rend la matrice $\mathbf{X}^T\mathbf{X}$ mal conditionnée et favorise le surapprentissage.

   Phénomène visé. Ce jeu illustre l'effet de la surparamétrisation : le modèle OLS s'adapte parfaitement aux données d'entraînement mais généralise mal.

3. **Jeu C : multicolinéarité.**

   Construction. On fixe $n = 1000$ et $d = 20$. On génère un vecteur latent $\mathbf{z} \sim \mathcal{N}(0, 1)$, puis deux premières colonnes fortement corrélées via $x_{i1} = z_i + \eta_{i1}$ et $x_{i2} = z_i + \eta_{i2}$ avec $\eta_{i1}, \eta_{i2} \sim \mathcal{N}(0, 0{,}1^2)$. Les autres colonnes sont tirées de manière indépendante suivant $\mathcal{N}(0, 1)$. Les coefficients $\mathbf{w}$ et les bruits ϵ_i sont choisis comme précédemment.

   Phénomène visé. Ce jeu met en évidence l'instabilité de l'estimateur OLS en présence de variables corrélées et l'effet stabilisateur de la régularisation.

Rappel des principaux algorithmes d'optimisation

Avant de réaliser les expériences, vous devez implémenter plusieurs algorithmes permettant de minimiser la fonction $L(\mathbf{w})$. On note L la fonction de coût, $\nabla L(\mathbf{w})$ son gradient évalué en $\mathbf{w}$ et $\epsilon > 0$ le pas d'apprentissage.

Descente de gradient. À l'itération k :

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \epsilon \, \nabla L\big(\mathbf{w}^{(k)}\big).$$

Descente de gradient avec moment. On introduit une vitesse $\boldsymbol{v}_t$ qui accumule les gradients :

$$\begin{aligned} \mathbf{m}^{(k+1)} &= \gamma \mathbf{m}^{(t)} + (1-\gamma) \nabla L(\mathbf{w}^{(k)}) \,, \\ \mathbf{w}^{(k+1)} &= \mathbf{w}^{(k)} - \epsilon \mathbf{m}^{(k+1)} \,. \end{aligned}$$

Le paramètre $\gamma \in [0, 1)$ contrôle l'inertie des pas successifs.

Descente de gradient accélérée (FISTA / Nesterov). On maintient deux suites d'itérés $\mathbf{w}^{(k)}$ et $\overline{\mathbf{w}}^{(k)}$ et une suite de coefficients λ_k. On initialise $\lambda_0 = 0$ et $\overline{\mathbf{w}}^{(0)} = \mathbf{w}^{(0)}$. Les mises à jour sont :

$$\begin{aligned} \mathbf{w}^{(k+1)} &= \overline{\mathbf{w}}^{(k)} - \epsilon \, \nabla L\big(\overline{\mathbf{w}}^{(k)}\big), \\ \lambda_{k+1} &= \frac{1 + \sqrt{1 + 4\,\lambda_k^2}}{2}, \qquad \eta_{k+1} = \frac{\lambda_k - 1}{\lambda_{k+1}}, \\ \overline{\mathbf{w}}^{(k+1)} &= \mathbf{w}^{(k+1)} + \eta_{k+1}\big(\mathbf{w}^{(k+1)} - \mathbf{w}^{(k)}\big). \end{aligned}$$

Exercices

Munis des algorithmes précédents, vous allez maintenant étudier empiriquement les phénomènes d'overfitting et de multicolinéarité sur les jeux de données présentés.

1. **Solution analytique**
   (a) Montrez que la solution analytique de la régression de Ridge s'écrit :

   $$\boldsymbol{w}_{\text{Ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda n \mathbf{I}_d)^{-1}\mathbf{X}^T\mathbf{y}\,.$$

   Pour $\lambda = 0$, on retrouve l'estimateur OLS.
   (b) Implémentez une fonction qui calcule cette solution pour $\mathbf{X}$ et $\mathbf{y}$ donnés.
2. **Gradient de la fonction objective**
   (a) Calculez analytiquement le gradient $\nabla L(\mathbf{w})$ de la fonction objective par rapport $\mathbf{w}$.
   (b) Implémentez en Python une fonction qui renvoit la valeur de $L(\mathbf{w})$ et de son gradient pour des matrices $\mathbf{X}$ et un vecteur $\mathbf{y}$ de dimensions arbitraires.
3. **Algorithmes d'optimisation**
   (a) Implémentez une fonction qui réalise la descente de gradient.
   (b) Implémentez une fonction qui réalise la descente de gradient avec moment.
   (c) Implémentez une fonction qui réalise la descente de gradient accélérée.
4. **Effet de la régularisation sur un modèle bien conditionné (Jeu A).**
   (a) Implémentez une fonction qui génère des observations du jeu de données A.
   (b) Séparez les 500 premières observations pour l'entraînement et les 500 restantes pour la validation.
   (c) Pour une valeur de λ donnée, appliquez la descente de gradient et analyser la convergence de l'optimisation. Pour cela, identifiez les hyperparamètres sur lesquels vous pouvez jouer pour contrôler la convergence de l'algorithme. Faites votre analyse en traçant l'évolution de la fonction de perte en fonction du nombre d'itération pour plusieurs valeurs des hyperparamètres.
   (d) Est-ce que la valeur de λ affecte la vitesse de convergence ?
   (e) Arrivez-vous à obtenir la solution analytique $\mathbf{w}_{\text{Ridge}}$?
   (f) Pour différentes valeurs de $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$, appliquez la descente de gradient et tracez : (i) les chemins de régularisation $\lambda \mapsto \beta_j$ pour $j = 1, \ldots, d$ et (ii) les MSE d'entraînement et de validation en fonction de λ. Commentez l'évolution de ces quantités et dites si la régularisation améliore la généralisation dans ce cas.
5. **Surparamétrisation et surapprentissage (Jeu B).**
   (a) Implémentez une fonction qui génère des observations du jeu de données B.
   (b) Séparez l'échantillon en un ensemble d'entraînement de taille 80 et un ensemble de test de taille 20. Calculez et tracez les valeurs de l'estimateur w_{Ridge} pour plusieurs valeurs de λ (0, 10, 100, 1000, ...). Quels phénomènes observe-t-on ?
   (c) Pour plusieurs valeurs de λ, utilisez la descente de gradient pour calculer l'estimateur de Ridge et tracez les MSE d'entraînement et de test en fonction de λ.
   (d) Comparez l'optimisation par descente de gradient classique, avec moment et FISTA en termes de vitesse de convergence et de qualité de solution obtenue.
6. **Multicolinéarité (Jeu C).**
   (a) Implémentez une fonction qui génère des observations du jeu de données C.
   (b) Appliquez la descente de gradient pour plusieurs valeurs de λ. Comparez les chemins de régularisation et les MSE à ceux obtenus sur le jeu A. Montrez que la régularisation stabilise les coefficients associés aux variables corrélées.
   (c) Comparez également les versions avec moment et FISTA pour illustrer l'impact des méthodes d'optimisation sur cette situation.